

NLP Journey

A step-by-step journey through Natural Language Processing

Table of Contents

Topic 1: Text Preprocessing

1. Tokenization (word-level, sentence-level, subword)
2. Stemming vs Lemmatization
3. Stopwords removal
4. Text cleaning/normalization (lowercasing, punctuation, special chars)

Topic 2: Text Representation (Basic)

1. One-hot encoding
2. Bag of Words (BoW)
3. TF-IDF (Term Frequency–Inverse Document Frequency)
4. N-grams as features

Topic 3: Word Embeddings

1. Word2Vec — CBOW
2. Word2Vec — Skip-gram
3. Negative Sampling
4. GloVe (co-occurrence based)

Topic 4: Sequence Modeling Context

1. Why BoW/TF-IDF lose word order
2. Fixed-window context limitations
3. Bridge to sequential models (RNN concept, no deep math)

Topic 5: NLP Tasks (Classical)

1. POS Tagging
2. Named Entity Recognition (NER)
3. Text Classification
4. Sentiment Analysis

Topic 6: Language Models (N-gram)

1. Unigram/Bigram/Trigram models
2. Markov assumption
3. Smoothing techniques (Laplace, etc.)
4. Perplexity as evaluation

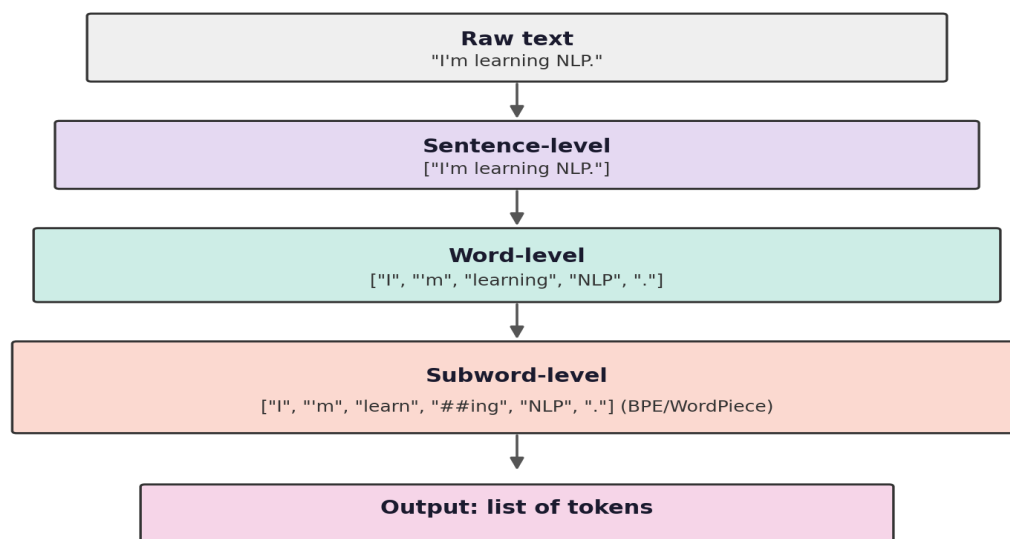
Topic 7: Evaluation Metrics

1. BLEU score
2. ROUGE score
3. Perplexity (tied back from #6)
4. Precision/Recall/F1 for classification tasks

TOPIC 1 OF 7

Text Preprocessing

Tokenization (word-level, sentence-level, subword)



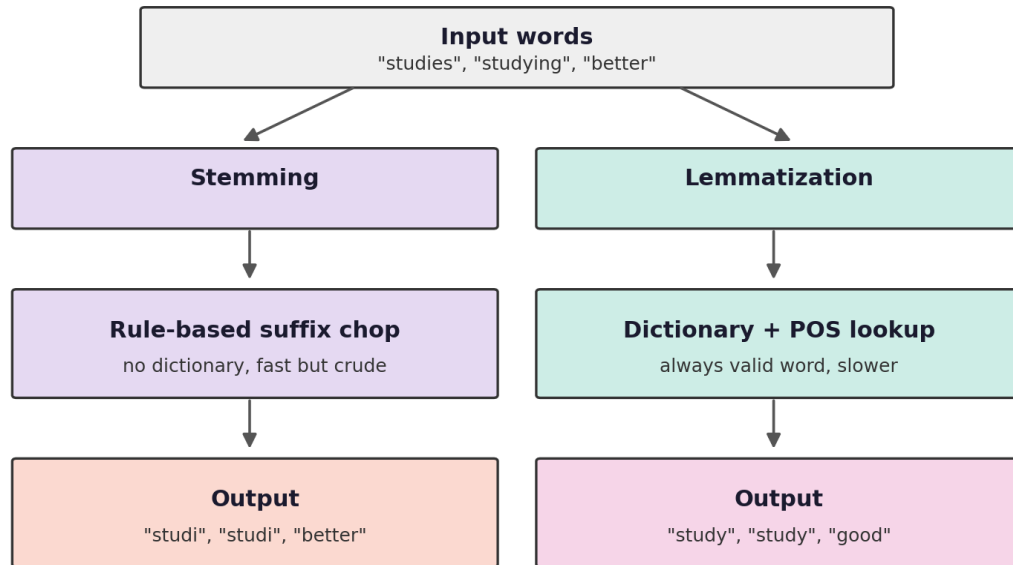
Core idea

Raw text is just a string of characters with no structure. Tokenization breaks it into meaningful units (tokens) so every later step has discrete pieces to work with.

Why it matters

Every downstream step — counting, embeddings, model input — needs discrete units. Sentence-level splits by sentence boundary, word-level splits by word/punctuation, and subword-level (BPE/WordPiece) breaks rare or long words into smaller known pieces, which is what modern Transformer/LLM tokenizers use to keep vocabulary size manageable.

Stemming vs Lemmatization



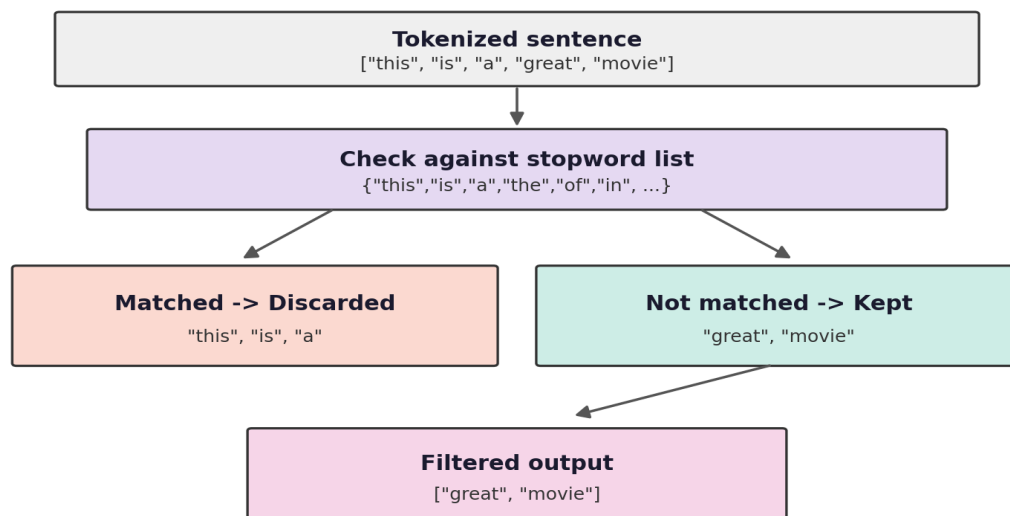
Core idea

Words like study, studies, studying share the same core meaning. Reducing them to one root form shrinks vocabulary and groups related words together instead of treating each form as unrelated.

Why it matters

Stemming is rule-based suffix chopping — fast, but output may not be a real word (studies → studi). Lemmatization uses a dictionary and part-of-speech info to return a valid base word (studies → study, better → good) — slower but accurate. Use stemming when speed matters, lemmatization when precision matters.

Stopwords removal



Core idea

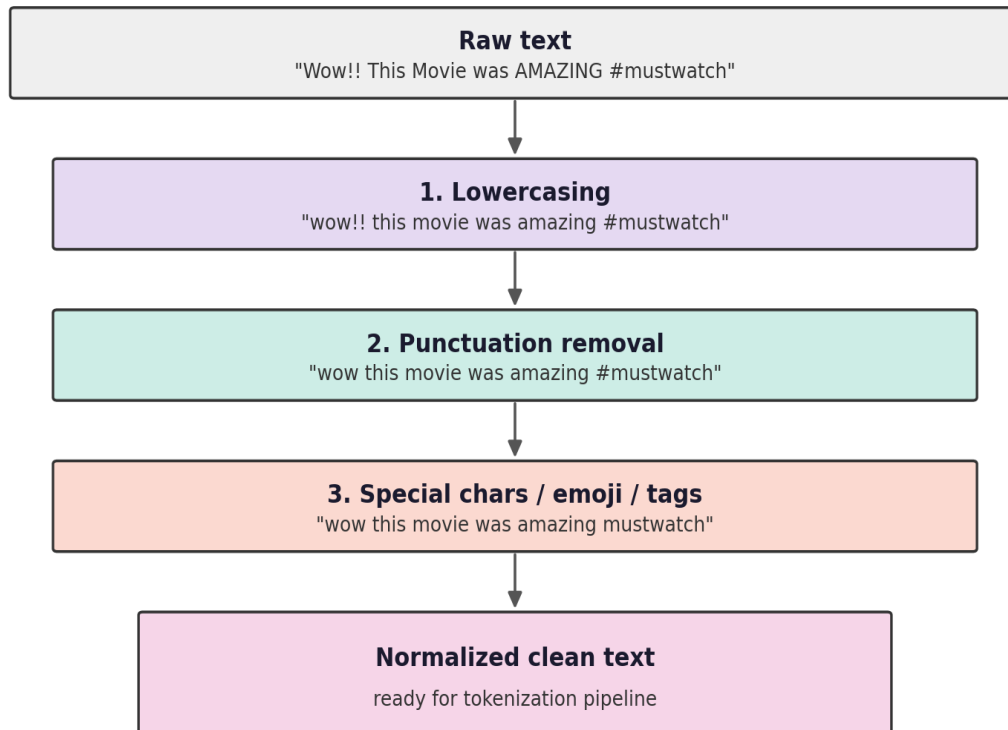
Words like this, is, a appear very frequently but carry little topic-specific meaning. Stopword removal filters these out so the remaining tokens carry more signal.

Why it matters

Every token is checked against a stopwords list; matches get discarded, the rest are kept. This reduces noise and vocabulary size before representation steps like Bag of Words or TF-IDF, where high-frequency low-information words would otherwise dominate the counts.

SUBTOPIC 4 OF 4

Text cleaning / normalization (lowercasing, punctuation, special chars)



Core idea

Raw text from real sources (tweets, reviews, chats) is messy — mixed case, punctuation, emojis, hashtags. Normalization standardizes it into a consistent, clean format.

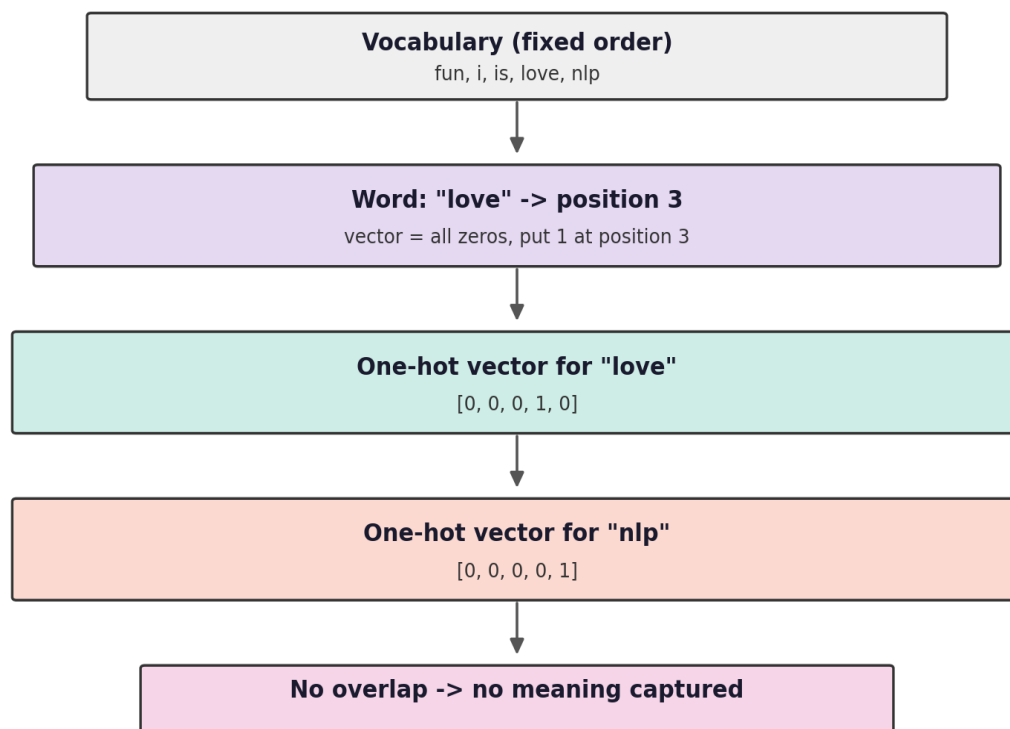
Why it matters

Lowercasing prevents Movie and movie being treated as different tokens. Punctuation removal drops symbols that usually add no meaning for classical NLP tasks. Special character/emoji/tag handling strips or converts noisy elements. The result is clean, normalized text ready to enter the tokenization pipeline.

TOPIC 2 OF 7

Text Representation (Basic)

One-hot encoding



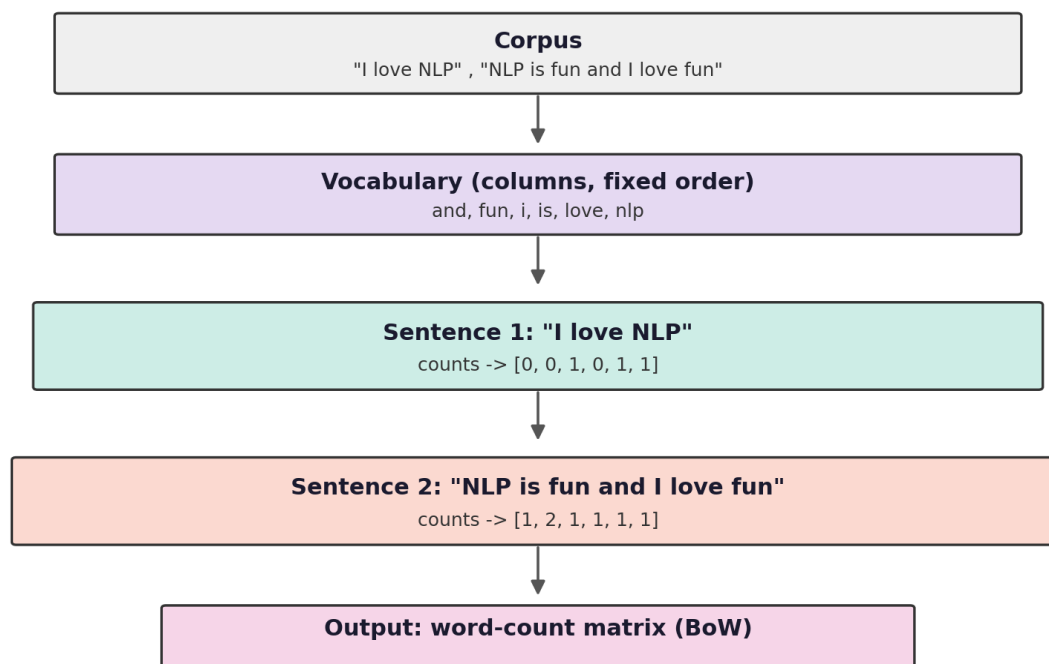
Core idea

Each unique word in the vocabulary gets a vector of zeros with a single 1 marking its own fixed position — a way to represent identity, nothing more.

Why it matters

Word love and word nlp share zero overlap in their vectors even though a model might want to know they're related — one-hot has no concept of meaning or similarity. Vectors also grow huge and mostly-empty as vocabulary grows. This limitation is exactly what Bag of Words and later Word Embeddings fix.

Bag of Words (BoW)



Core idea

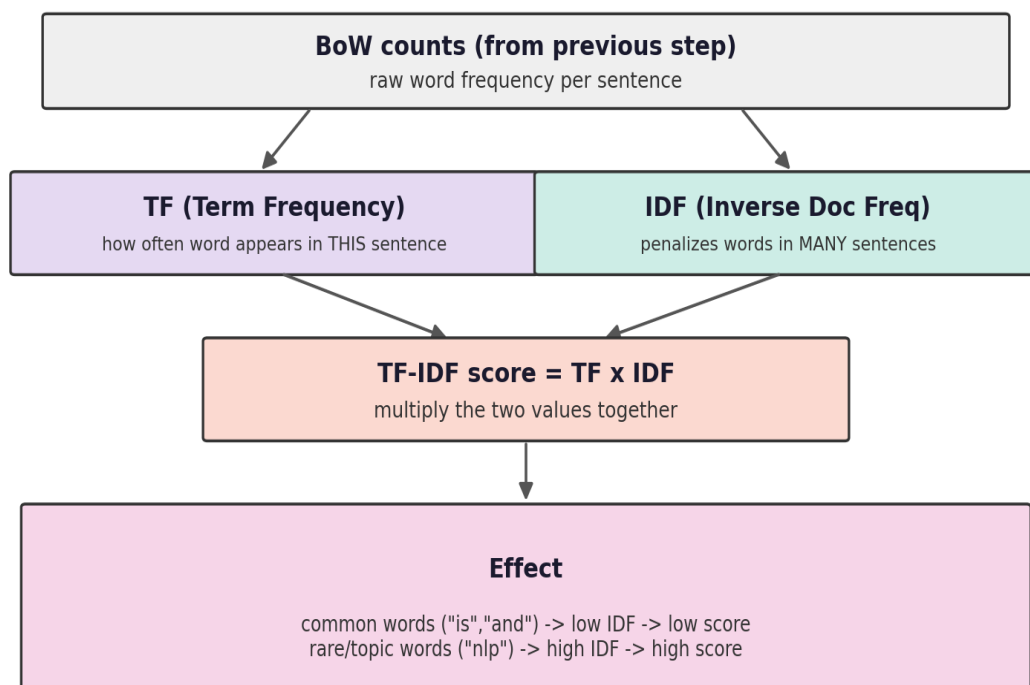
Instead of one vector per word, BoW gives one vector per sentence. Each position corresponds to a fixed vocabulary word, and the value is how many times that word appears in the sentence.

Why it matters

Frequency now matters — a word appearing twice gets counted as 2, not just marked present. But word order is still lost (it's a 'bag'), and very common words like is, and can dominate counts even though they carry little meaning — the exact problem TF-IDF solves next.

SUBTOPIC 3 OF 4

TF-IDF (Term Frequency–Inverse Document Frequency)



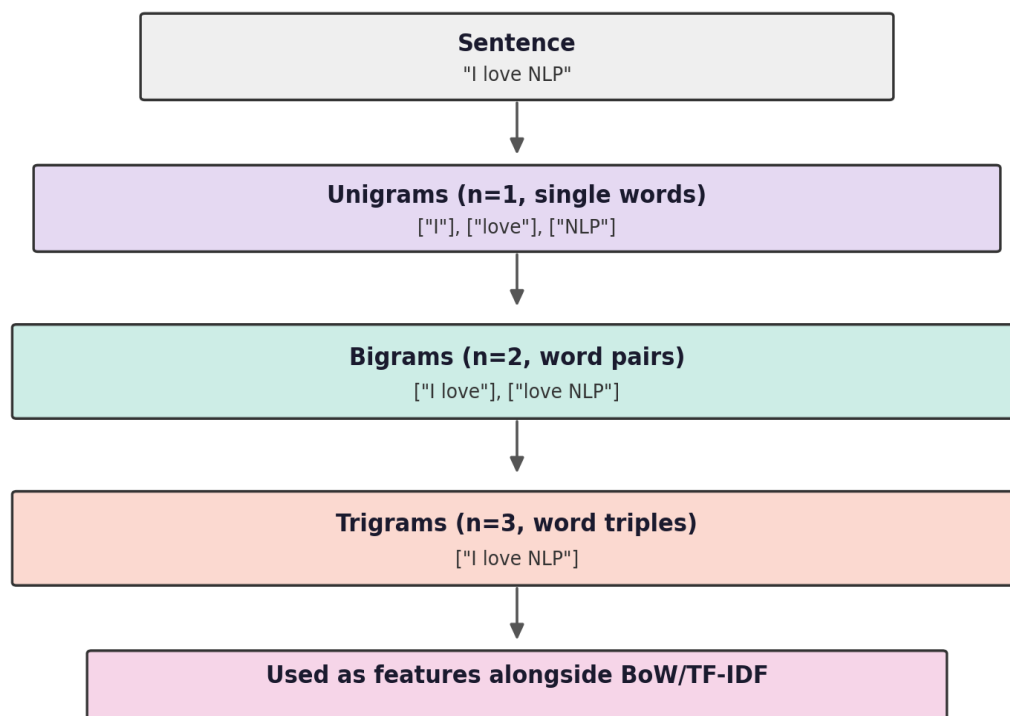
Core idea

TF-IDF reweights BoW counts using two parts multiplied together: TF (how often a word appears in this sentence) and IDF (a penalty for words that appear in many sentences across the corpus).

Why it matters

Common filler words like *is*, *and* appear in almost every sentence, so IDF pushes their score down. Rare, topic-specific words like *nlp* appear in only a few sentences, so IDF pushes their score up. Final score = TF x IDF, so meaningful words stand out and filler words get suppressed.

N-grams as features



Core idea

Instead of treating single words as features (unigrams), n-grams group consecutive words together — bigrams (2 words), trigrams (3 words) — before counting or weighting with BoW/TF-IDF.

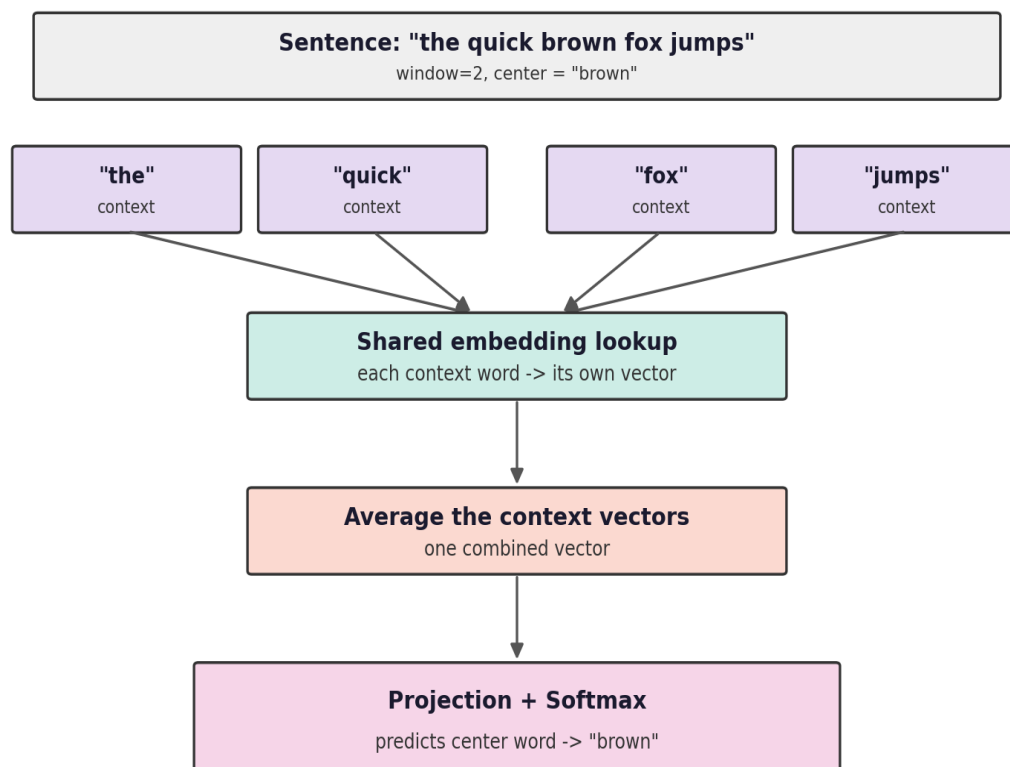
Why it matters

Single-word counting loses connections between words — 'not' and 'good' become two separate, unrelated counts, so 'not good' and 'good' look similar. Bigrams turn 'not good' into one single feature, distinct from 'good' alone, capturing local word order and relationships like negation. Trade-off: bigger n captures more context but grows the vocabulary fast.

TOPIC 3 OF 7

Word Embeddings

Word2Vec — CBOW (Continuous Bag of Words)



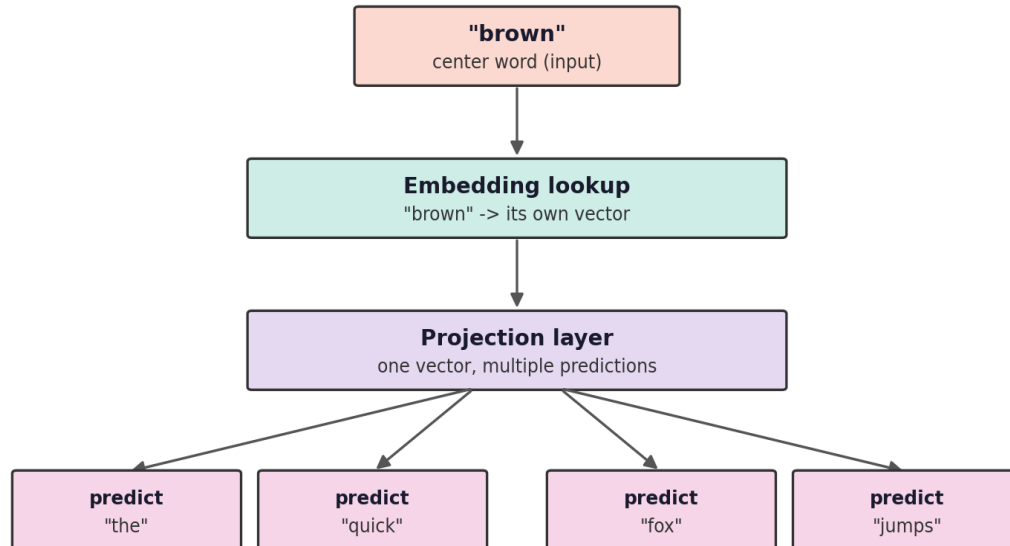
Core idea

CBOW takes the surrounding context words (a fixed window on both sides of a target word) and tries to predict the missing center word. The context word vectors are averaged into one combined vector before prediction.

Why it matters

The prediction task itself isn't the goal — it's a training exercise. To get good at this fill-in-the-blank task across millions of sentences, the model is forced to learn embedding vectors where words appearing in similar contexts end up with similar vectors. Those learned vectors, not the prediction accuracy, are the real deliverable.

Word2Vec — Skip-gram



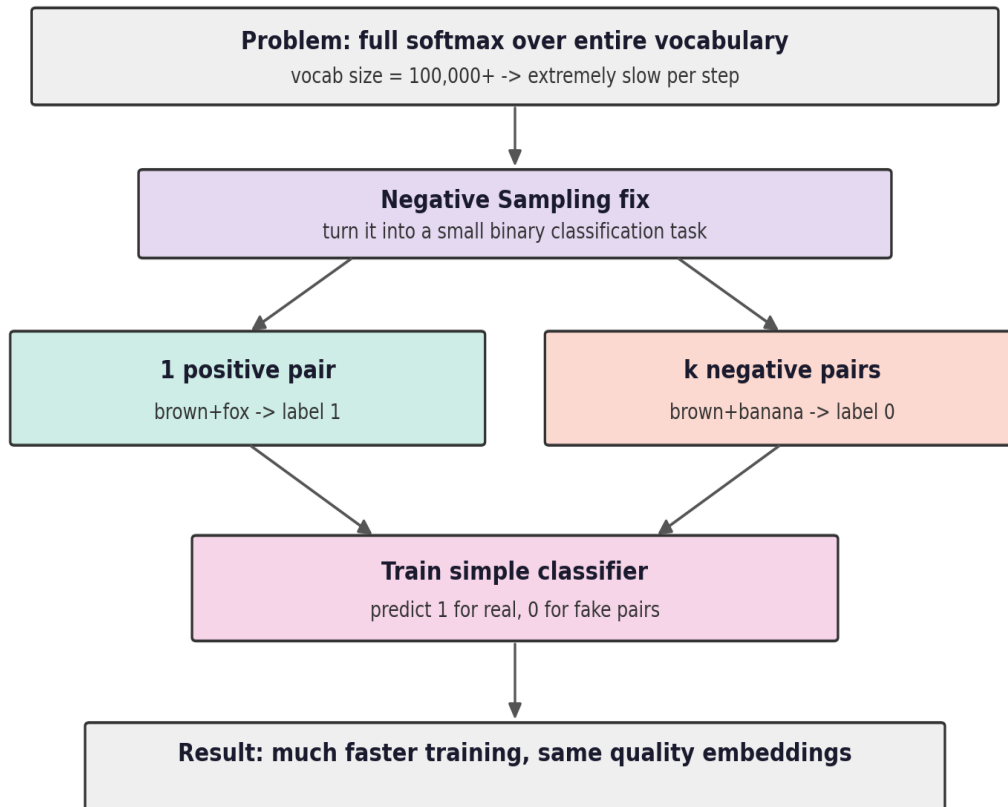
Core idea

Skip-gram is the mirror image of CBOW: instead of predicting the center word from context, it takes the center word and tries to predict each surrounding context word separately.

Why it matters

Skip-gram generates multiple training examples from a single center word (one per context word), which tends to work better for rare words. CBOW averages context into one prediction and trains faster. Both exist because of this speed vs. rare-word-quality trade-off.

Negative Sampling



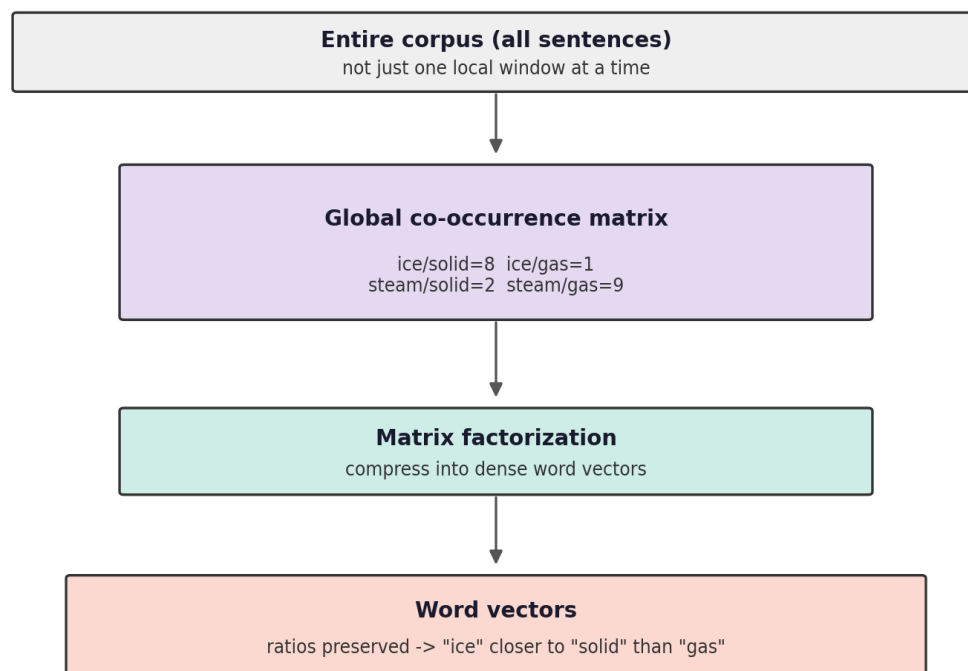
Core idea

Training with a full softmax over a huge vocabulary (100,000+ words) is extremely slow, since every step compares against every word. Negative sampling turns this into a small binary classification task instead.

Why it matters

Each training step only uses one real (positive) pair, labeled 1, plus a handful of random (negative) pairs, labeled 0 — both are used in training so the model learns the difference, not just always predicting 1. This means only a few words are compared per step instead of the entire vocabulary, making training dramatically faster with minimal quality loss.

GloVe (co-occurrence based)



Core idea

Unlike Word2Vec, which learns from small local windows one at a time, GloVe first counts how often every pair of words appears together across the entire corpus, building one global co-occurrence matrix.

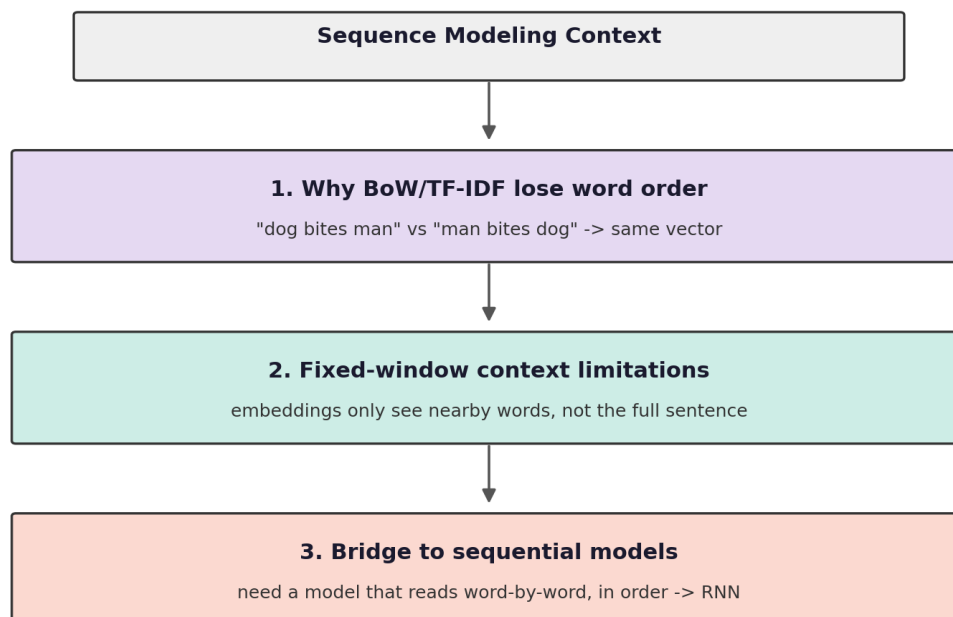
Why it matters

GloVe's key insight is that ratios of co-occurrence counts reveal word relationships (e.g. ice appears with solid far more than with gas). The matrix is then factorized/compressed into dense word vectors that preserve these ratios. Same goal as Word2Vec — meaningful word vectors — reached via global statistics instead of local prediction.

TOPIC 4 OF 7

Sequence Modeling Context

Why word order and fixed windows fall short



1. Why BoW/TF-IDF lose word order

"Dog bites man" and "man bites dog" contain the exact same words, so BoW/TF-IDF produce the exact same vector for both — even though the meaning is completely different. Counting words ignores the order they appear in.

2. Fixed-window context limitations

Word2Vec (CBOW/Skip-gram) only looks at a small fixed window (e.g. 2 words on each side). In a long sentence, a word far outside that window can never be connected to the target word, even if it's actually relevant to its meaning.

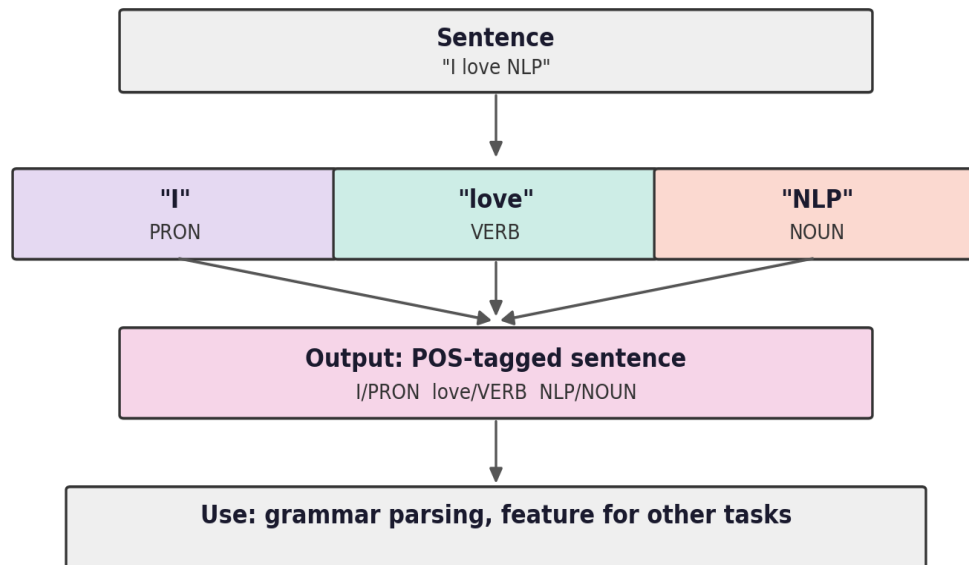
3. Bridge to sequential models (RNN concept)

Both problems point to the same fix: a model that reads text word-by-word in order, carrying forward memory of everything read so far instead of only a fixed nearby window. This is the motivation behind Recurrent Neural Networks (RNN), covered in depth later in the roadmap.

TOPIC 5 OF 7

NLP Tasks (Classical)

POS Tagging (Part-of-Speech Tagging)



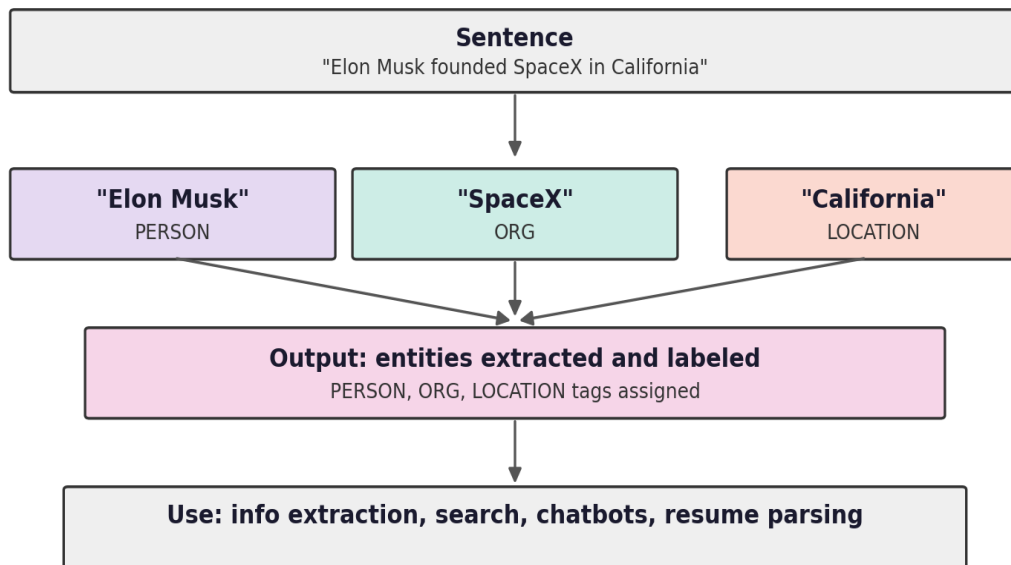
Core idea

POS tagging labels every word in a sentence with its grammatical role — pronoun, verb, noun, adjective, and so on. "I love NLP" becomes I/PRON, love/VERB, NLP/NOUN.

Why it matters

Knowing a word's grammatical role helps disambiguate meaning and structure sentences correctly. It's used as a building block for parsing and as an extra feature for other NLP tasks.

Named Entity Recognition (NER)



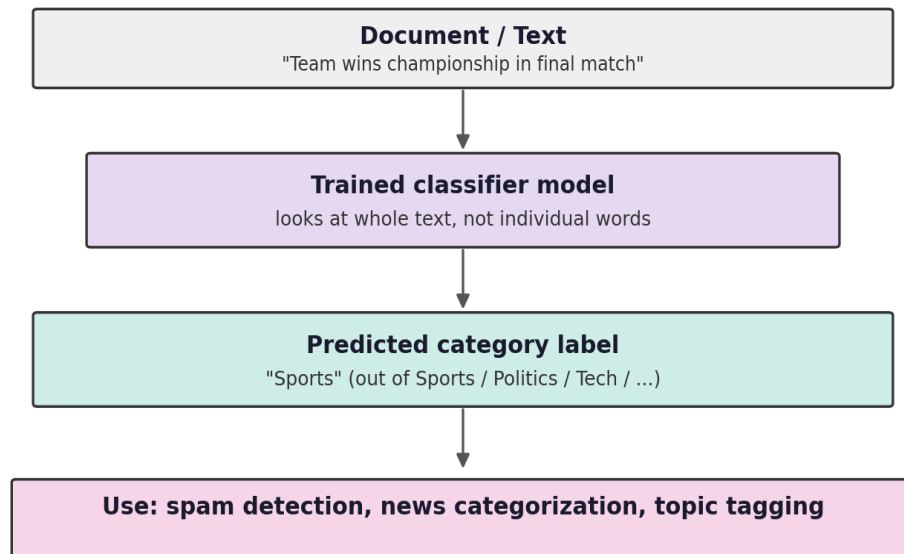
Core idea

NER scans text and identifies real-world entities — people, organizations, locations, dates — then labels each one. "Elon Musk founded SpaceX in California" tags Elon Musk as PERSON, SpaceX as ORG, California as LOCATION.

Why it matters

This is how systems extract structured facts from unstructured text — powering search engines, resume parsers, chatbots, and information extraction pipelines.

Text Classification



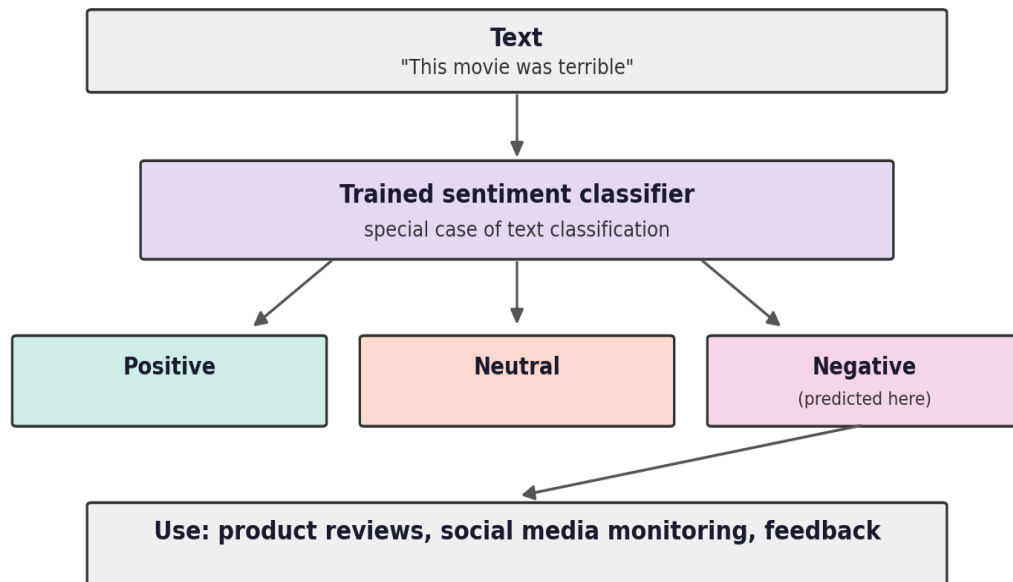
Core idea

A trained classifier looks at an entire piece of text and assigns it one category label out of a predefined set, e.g. Sports, Politics, Tech.

Why it matters

This is the foundation behind spam detection, news categorization, and topic tagging — anywhere a whole document needs a single label rather than word-by-word tags.

Sentiment Analysis



Core idea

Sentiment analysis is a specific case of text classification where the categories are emotional tone: Positive, Neutral, or Negative. "This movie was terrible" is classified as Negative.

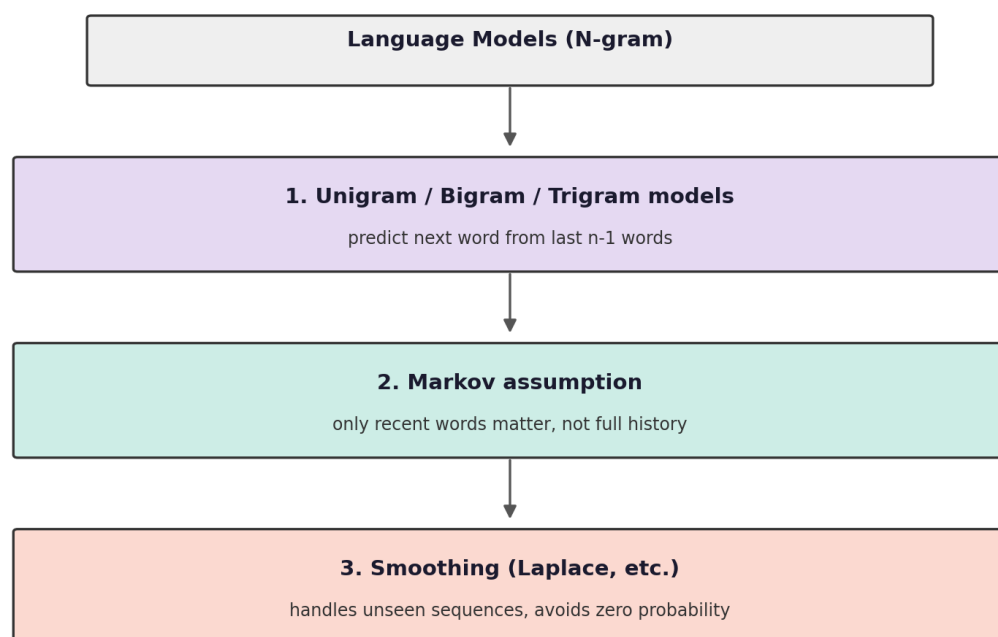
Why it matters

Widely used for product review analysis, social media monitoring, and customer feedback processing — turning large volumes of text into a simple, actionable signal.

TOPIC 6 OF 7

Language Models (N-gram)

Unigram, Bigram, Trigram, Markov Assumption, Smoothing



1. Unigram / Bigram / Trigram models

These models try to predict the next word using the previous $n-1$ words. A bigram model looks only at the last 1 word to guess the next; a trigram model looks at the last 2. These are the same n -gram units from Topic 2, now used for prediction instead of counting.

2. Markov assumption

The core simplification that makes n -gram models practical: to predict the next word, you don't need the entire sentence history — only the last few words matter. This assumption trades some accuracy for speed and simplicity.

3. Smoothing techniques (Laplace, etc.)

If a word sequence never appeared in training data, the model would assign it zero probability — treating it as impossible rather than just rare. Smoothing assigns a small non-zero probability to every possible sequence so the model can still handle unseen combinations gracefully.

Perplexity as evaluation

4. Perplexity as evaluation
lower perplexity -> better model

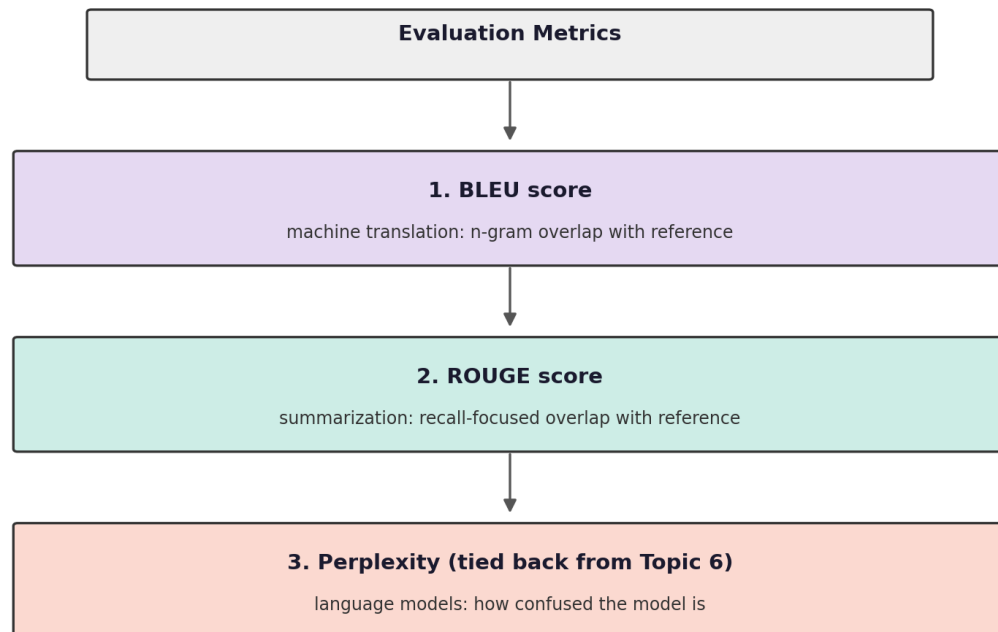
Core idea

Perplexity measures how 'confused' or 'surprised' a language model is when predicting the next word in real text. A lower perplexity means the model assigns higher probability to the words that actually appear — i.e. it predicts better.

Why it matters

Perplexity is the standard way to compare language models numerically, without needing human judgment. It connects directly to Topic 7's evaluation metrics, where it's also used to evaluate modern language models.

BLEU, ROUGE, and Perplexity



1. BLEU score

Used to evaluate machine translation. Compares the model's translated output against one or more human reference translations by measuring n-gram overlap. Higher overlap means a higher, better BLEU score.

2. ROUGE score

Used to evaluate summarization. Similar overlap idea to BLEU, but recall-focused — it checks how much of the important content from the reference summary is captured in the model's output.

3. Perplexity (tied back from Topic 6)

Used to evaluate language models — how confused the model is when predicting the next word. Lower perplexity means better predictive performance.

Precision / Recall / F1 for classification tasks

4. Precision / Recall / F1

classification: correctness vs completeness of predictions

Core idea

Used to evaluate classification tasks like sentiment analysis or NER. Precision measures how many of the model's positive predictions were actually correct (avoiding false alarms). Recall measures how many of the actual positives the model successfully found (avoiding misses). F1 is the harmonic mean of the two, balancing both concerns into a single score.

Why it matters

Accuracy alone can be misleading on imbalanced data (e.g. rare disease detection, fraud detection). Precision, Recall, and F1 give a fuller picture of a classifier's real-world performance, and are still standard for evaluating classification-style outputs from modern models.